DESARROLLO BACKEND

Observaciones sobre el Diseño Técnico y lo que ya implementamos

Marcos Teixido · German Velazquez · Axel Nievas — 12 de octubre de 2026

Para qué es este documento

Antes de programar revisamos el **diagrama de clases**, los **diagramas de actividad** (Nueva partida, Realizar jugada, Turno de máquina), la tabla de **endpoints** de Diseño Técnico, el **modelo de datos**, la **arquitectura** y los **requisitos**. Encontramos puntos que faltan o que se contradicen entre documentos.

Para no frenar el desarrollo (ya pasó el 60 % del tiempo), **aplicamos en el backend una solución para cada punto** y la dejamos documentada acá. **No modificamos ningún documento ni trabajo de otro grupo**: cada responsable decide si adopta la solución o propone otra, y si cambia algo, ajustamos el código.

La sección 1 resume los 14 puntos y quién los revisa; la 2 los detalla (qué dice hoy, por qué no cierra, qué hizo Backend y qué pedimos); la 3 muestra el avance del backend y la 4, lo que necesitamos de cada grupo.

1. Resumen

#	Punto	Revisa	Propuesta de backend
1	Faltan en el diagrama las clases de servicio y dominio (GestorDePartida, Tablero, etc.)	Diseño Técnico	Implementadas
2	La partida no tiene estado (En Curso / Pendiente / Finalizada) ni motivo de fin	Datos + Diseño Técnico	Propuesto
3	No se guarda el nombre (alias) de los jugadores — corrección C4	Datos + Requisitos	Propuesto
4	DifficultyProfile no tiene rango ELO	Datos + Arquitectura	Propuesto (falta confirmar ELO)
5	getNextMove de la IA tiene otra firma que la interfaz	Diseño Técnico	Resuelto
6	El fallback de la IA a Stockfish no está en el diagrama	Diseño Técnico + Simulación	Resuelto
7	Faltan métodos en los repositorios (historial, pausa, importar)	Diseño Técnico + Requisitos	Parcial (falta importar)
8	La jugada no se guarda en formato UCI (el del dataset)	Datos + Simulación	Propuesto
9	La IA se entrena también con partidas no finalizadas	Arquitectura + Simulación	Propuesto (vista)
10	Endpoints y puerto distintos entre Arquitectura y Diseño Técnico	Arquitectura	Seguimos Diseño Técnico
11	Nombres de estados mezclados entre documentos	Todos	Unificado
12	Arquitectura pide que las partidas interrumpidas NO se guarden	Arquitectura + QA	Seguimos RN-01
13	Base local vs. Supabase y contraseña escrita en la guía	GC	A decidir por GC
14	Los motores y entidades tienen nombres distintos en diagrama, manual y carpetas del repo	Diseño Técnico + Simulación	Seguimos el diagrama

2. Detalle de cada punto

1 Faltan las clases de servicio y de dominio

Diseño Técnico

Qué dice hoy	La tabla de endpoints nombra **GestorDePartida**, **ServicioHistorial** y el manual de roles **ValidadorDeJugada**; Arquitectura nombra **Tablero** (fachada de chess.js) y **GameSessionFactory**. El diagrama de clases solo tiene entidades, repositorios y oponentes.
Por qué no cierra	Son el núcleo del backend (reglas, turnos, estados). Si no están diseñadas, el programador termina decidiendo su forma, que es justo lo que el profesor pidió evitar (C3).
Qué hizo Backend	Las implementamos con esos nombres: **Tablero** (legalidad RN-02 Bis y fin RN-03), **ValidadorDeJugada** (forma de los datos de entrada), **GestorDePartida** (CU-01 a CU-04, CU-06, pausa, abandono), **ServicioHistorial** (CU-05 y PGN) y **GameSessionFactory** (elige el oponente).
Qué pedimos	Agregarlas al diagrama de clases. Les pasamos los métodos públicos de cada una (están en el código y en el README).

2 La partida no tiene estado ni motivo de fin

Datos + Diseño Técnico

Qué dice hoy	Game tiene `result` pero no un estado. RN-01 define tres estados: En Curso, Pendiente/Pausada y Finalizada. CU-06 pide declarar el resultado "con su motivo reglamentario".
Por qué no cierra	Sin estado no se puede pausar, reanudar ni filtrar el historial por pendientes. Sin motivo, la pantalla de resultado no puede decir si fue mate, ahogado, triple repetición, etc.
Qué hizo Backend	Agregamos a `games`: `status` (IN_PROGRESS, PAUSED, FINISHED) y `end_reason` (CHECKMATE, STALEMATE, THREEFOLD_REPETITION, FIFTY_MOVES, INSUFFICIENT_MATERIAL, RESIGNATION). Una restricción de la base impide que una partida quede FINISHED sin resultado, motivo y fecha de fin.
Qué pedimos	Que Datos los incorpore al modelo físico y Diseño Técnico al diagrama.

3 No se guarda el nombre de los jugadores (C4)

Datos + Requisitos

Qué dice hoy	CU-01 pide el alias y `POST /games` lo recibe, pero Game no tiene dónde guardarlo. El color sí se deduce de `whiteType` y `blackType`.
Por qué no cierra	El profesor marcó en la clase del 30/9 que faltaban nombres, color y dificultad en el guardado. El historial (CU-05) también debe mostrar el alias.
Qué hizo Backend	Agregamos `white_alias` y `black_alias` (dos campos porque en el modo local juegan dos personas). Contra la máquina, el lado de la máquina se guarda como "Stockfish" o "IA propia". El historial permite filtrar por alias.
Qué pedimos	Que Datos lo sume al modelo. Que Requisitos aclare en CU-01 si en modo local se piden los dos nombres (hoy dejamos ambos opcionales: si no se cargan, quedan "Blancas" y "Negras").

4 DifficultyProfile no tiene rango ELO

Datos + Arquitectura

Qué dice hoy	Requisitos asocia cada dificultad a un rango ELO (800–1200, 1300–1700, 1800+) y `/predict-move` recibe `elo`. La tabla de perfiles no tiene ELO. Arquitectura configura Stockfish por Skill Level (3 / 10 / 20).
Por qué no cierra	No hay de dónde sacar el ELO que la IA necesita ni el que la pantalla de dificultad muestra. Además sigue abierta la duda "¿ELO o nivel?" que Simulación y Coordinación anotaron para el profesor.
Qué hizo Backend	Agregamos `elo_min` y `elo_max`, y sembramos 6 perfiles (Fácil/Medio/Difícil para Stockfish y para la IA). Stockfish usa el Skill Level de Arquitectura. El código solo lee la tabla: si el profesor decide otra cosa, se cambian los valores y no el código.
Qué pedimos	Confirmar con el profesor si el ELO se usa o es solo informativo.

5 La firma de getNextMove no coincide

Diseño Técnico

Qué dice hoy	OpponentStrategy define `getNextMove(fen)`, pero CustomAIOpponent tiene `getNextMove(fen, gameId)`.
Por qué no cierra	Con firmas distintas, Backend no puede usar los oponentes "indistintamente" (patrón Strategy).
Qué hizo Backend	Unificamos: `getNextMove(fen, context)` en todos, donde context lleva `{ gameId, profile }`. Todos devuelven la jugada en formato UCI ("e2e4", "e7e8q").
Qué pedimos	Actualizar la firma en el diagrama.

6 El respaldo de la IA a Stockfish no está en el diagrama

Diseño Técnico + Simulación

Qué dice hoy	Diseño Técnico decidió que si `/predict-move` responde `source: "none"`, la IA delega en Stockfish con Skill Level mínimo, desde Node. El diagrama de actividad "Turno de máquina" lo muestra, pero en el de clases CustomAIOpponent solo tiene el adaptador de IA.
Por qué no cierra	Sin un Stockfish disponible dentro de CustomAIOpponent, el respaldo no se puede ejecutar.
Qué hizo Backend	CustomAIOpponent recibe además un StockfishOpponent de respaldo (Skill Level 0, 300 ms). Está probado: con `source: "none"` la jugada sale de Stockfish.
Qué pedimos	Agregar la relación al diagrama. Ojo: el texto de Arquitectura (6.2) todavía dice que el respaldo es "material o jugada al azar"; conviene alinearlo con lo que decidió Diseño Técnico.

7 Faltan métodos en los repositorios

Diseño Técnico + Requisitos

Qué dice hoy	GameRepository tiene create, findById, updateCurrentFen y updateResult.
Por qué no cierra	No alcanza para el historial con filtros (CU-05), pausar/reanudar, ni para importar partidas, que el profesor pidió como caso de uso de soporte (C9).
Qué hizo Backend	Agregamos `findAll(filtros)` (modo, dificultad, estado, resultado, alias, paginado) y `updateStatus`. Exportar ya funciona con `GET /games/:id/pgn`. **Importar no lo implementamos** porque falta su caso de uso y su endpoint.
Qué pedimos	Requisitos: escribir los CU de importar y exportar. Diseño Técnico: fijar el endpoint de importar (proponemos `POST /games/import` con el texto PGN).

8 La jugada no se guarda en formato UCI

Datos + Simulación

Qué dice hoy	Move guarda SAN y los FEN. Simulación definió su dataset como FEN -> jugada en texto ("e2e4").
Por qué no cierra	El SAN ("Nf3", "exd6") hay que convertirlo para entrenar. Además, la IA necesita reconocer la misma posición aunque haya ocurrido en otra jugada (Arquitectura 6.2, "FEN simplificado").
Qué hizo Backend	Agregamos `uci`, y una columna calculada `fen_before_key` (los 4 primeros campos del FEN) con índice. También `UNIQUE(game_id, move_number)` para que dos pedidos simultáneos no guarden dos veces la misma jugada.
Qué pedimos	Que Datos lo incorpore y Simulación lo use para consultar posiciones.

9 La IA se entrena con partidas no finalizadas

Arquitectura + Simulación

Qué dice hoy	RN-01 y Arquitectura dicen que solo las partidas Finalizadas alimentan a la IA. Una nota de la misma pestaña de Arquitectura dice que el código actual no filtra por estado.
Por qué no cierra	Partidas abandonadas a la mitad o en curso contaminan las estadísticas de la IA.
Qué hizo Backend	Creamos la vista `v_training_moves`, que solo devuelve jugadas de partidas FINISHED, con su resultado y origen (USER/SEED). Está probada contra PostgreSQL.
Qué pedimos	Que el pipeline de Simulación (dataset_builder.py) lea de la vista en lugar de la tabla moves.

10 Endpoints y puerto distintos

Arquitectura

Qué dice hoy	Arquitectura: `/api/games`, `/api/history`, `POST /move` que devuelve también la jugada rival, puerto 4000. Diseño Técnico: `/games` sin prefijo, `POST /moves` y `POST /engine-move` separados. La guía de despliegue: puerto 3001.
Por qué no cierra	Frontend y QA no saben contra qué programar o probar.
Qué hizo Backend	Seguimos a **Diseño Técnico** (es quien fija los endpoints según el manual de roles) y a la **guía** (puerto 3001). La jugada de la máquina va separada: cada respuesta trae `machineTurn` y, si es true, el frontend llama a `engine-move`. Así un timeout del motor no pierde la jugada del humano.
Qué pedimos	Que Arquitectura actualice su sección 8 y el puerto. QA: el reporte PF-03 usa el puerto 4000.

11 Nombres de estados mezclados

Todos

Qué dice hoy	Aparecen "En Curso / Pendiente / Finalizada" (Requisitos), "EN_PROGRESO / FINALIZADA / INTERRUMPIDA" (Arquitectura) e "IN_PROGRESS" (nota de Arquitectura sobre el código).
Por qué no cierra	Si cada grupo usa un nombre distinto, el frontend, la base y la IA no se entienden.
Qué hizo Backend	Unificamos en inglés, igual que `mode` (LOCAL, STOCKFISH, AI): IN_PROGRESS = En Curso, PAUSED = Pendiente/Pausada, FINISHED = Finalizada. Resultados: WHITE_WINS, BLACK_WINS, DRAW.
Qué pedimos	Usar estos valores en todos los documentos y en el frontend.

12 Contradicción sobre guardar partidas interrumpidas

Arquitectura + QA

Qué dice hoy	RN-01 y Arquitectura (sección 7) piden guardar cada jugada al momento. Pero la sección 10 de Arquitectura pide "constatar que las partidas interrumpidas no inserten registros".
Por qué no cierra	Son opuestas: si se guarda jugada a jugada, una partida interrumpida sí tiene registros (queda Pendiente).
Qué hizo Backend	Seguimos RN-01: se guarda cada jugada, y la partida interrumpida queda PAUSED o IN_PROGRESS y **no entra al entrenamiento** (punto 9).
Qué pedimos	Corregir la prueba de la sección 10: lo que se verifica es que no alimenten a la IA, no que no se guarden.

13 Base local vs. Supabase, y contraseña en la guía

GC

Qué dice hoy	Arquitectura y GC hablan de PostgreSQL local (y Docker para pruebas). La Guía de Despliegue usa una base compartida en Supabase y tiene la contraseña escrita, aunque el mismo documento pide no compartirla.
Por qué no cierra	Si la guía circula, la base queda expuesta. Y hay que definir cuál es el entorno oficial de desarrollo y cuál el de pruebas.
Qué hizo Backend	El backend no tiene ninguna contraseña en el código: lee `DATABASE_URL` del `.env` (que está en .gitignore) y no arranca si falta. Dejamos un `.env.example` sin datos reales.
Qué pedimos	GC: decidir local o Supabase, y si la guía ya circuló, cambiar la contraseña de Supabase.

14 Nombres distintos entre diagrama, manual y repo

Diseño Técnico + Simulación

Qué dice hoy	El mismo motor se llama **StockfishOpponent** en el diagrama de clases, **MotorStockfish** en el manual de roles y **MotorExterno** en el README de `backend/src/engines`. La IA es **CustomAIOpponent** o **MotorIAPropia**. El README de `models` lista una entidad **Jugador/Perfil** que el diagrama no tiene. El de `validators` asigna las reglas del ajedrez a ValidadorDeJugada.
Por qué no cierra	Diseño Técnico fija los nombres definitivos y nadie más los cambia sin avisar. Con tres versiones, Backend y Simulación pueden programar la misma clase con nombres distintos.
Qué hizo Backend	Usamos los nombres del **diagrama de clases**. Las reglas viven en **Tablero** y la forma de los datos en **ValidadorDeJugada**, ambos en `validators/`. El jugador queda representado por el alias y el tipo (HUMAN, STOCKFISH, AI) dentro de la partida, sin tabla aparte. El código respeta las carpetas del repo (controllers, engines, models, routes, validators, tests) y suma services, repositories, config, db y middlewares.
Qué pedimos	Que Diseño Técnico confirme los nombres y se actualicen los README de las carpetas. Que Simulación y Backend acuerden quién mantiene cada archivo de `engines/`. Backend escribió un adaptador mínimo de Stockfish solo para poder probar (ver sección 3); Simulación decide si lo adopta o usa su propio MotorStockfish.

Cosas a tener en cuenta (decisiones técnicas)

No son errores del diseño, pero afectan a otros grupos:

- **Triple repetición:** necesita el historial de posiciones, que el FEN solo no tiene. Por eso al cargar o reanudar una partida el tablero se **reconstruye repitiendo todas las jugadas** guardadas, no desde el último FEN.
- **Transacción:** la jugada, el nuevo FEN y el cierre de la partida (si hubo mate o tablas) se guardan juntos. Si algo falla, no queda nada a medias.
- **Promoción:** si un peón llega a la última fila sin indicar la pieza, el backend responde `PROMOTION_REQUIRED`. El frontend debe mostrar el diálogo de Dama, Torre, Alfil o Caballo y reenviar la jugada.

- **Timeout del motor (CU-03 3a):** responde `504 ENGINE_TIMEOUT` con `retryable: true` y sin tocar el tablero. El frontend debe ofrecer "Reintentar" o "Pausar". Esto resuelve el defecto que QA describió en su reporte de ejemplo PF-03.
- **Abandono en modo local:** `POST /resign` recibe qué color se rinde. Contra la máquina solo puede rendirse el humano.

3. Qué implementó Backend

Está completa la **Fase 1** de los hitos (juego local con todas las reglas y persistencia) y funciona el modo **vs. Stockfish** (Fase 2). Backend no modificó el trabajo de Simulación: escribió solo un adaptador mínimo para poder probar, con lo que ya definía Arquitectura (6.1), y Simulación decide si lo adopta o usa el suyo. De la IA propia no se desarrolló nada: solo la llamada al servicio, que espera el trabajo de Simulación. El código está en la carpeta `backend/` con un README que documenta cada endpoint, su entrada, su salida y sus errores.

Método	Ruta	CU	Estado
GET	/health	–	Listo
GET	/difficulty-profiles	CU-01	Listo
POST	/games	CU-01	Listo
GET	/games (filtros: mode, difficulty, status, result, alias)	CU-05	Listo
GET	/games/:id (estado + jugadas)	CU-05	Listo
GET	/games/:id/pgn (exportar)	CU-05	Listo
GET	/games/:id/legal-moves?square=e2	CU-06	Listo
POST	/games/:id/moves	CU-06	Listo
POST	/games/:id/engine-move	CU-03 / CU-04	Stockfish: listo · IA: espera servicio Python
POST	/games/:id/pause · /resume	CU-02 a CU-05	Listo
POST	/games/:id/resign	RN-03	Listo
POST	/games/import (propuesto)	C9	Pendiente de CU y endpoint

Reglas cubiertas

- **RN-02 Bis:** legalidad completa (no dejar al propio rey en jaque, piezas clavadas), enroque corto y largo con casillas atacadas, captura al paso solo en el turno inmediato, y promoción a pieza elegida.
- **RN-03:** jaque mate, ahogado, triple repetición, regla de los 50 movimientos, material insuficiente y abandono.
- **RN-01:** persistencia de cada jugada (SAN, UCI, FEN antes y después) y ciclo de estados En Curso, Pendiente y Finalizada.

Pruebas

Tipo	Cantidad	Qué cubren
Unitarias de dominio	15	Tablero: cada regla de RN-02 Bis y RN-03, con posiciones concretas.
Validación de entrada	21	Una prueba por cada entrada válida e inválida de la API, y el fail-fast de configuración.

Tipo	Cantidad	Qué cubren
API (HTTP)	24	Flujos completos de CU-01 a CU-06: partida hasta el mate, pausa y reanudación, abandono, turno de la máquina, timeout, respaldo de la IA e historial con filtros y PGN.
Adaptador de Stockfish	11	Secuencia UCI, proceso reutilizado, cola de pedidos, timeout, caída y reinicio del motor, ruta mal configurada, y una partida contra Stockfish 17.1 real (incluye encontrar un mate en 1).
Integración con PostgreSQL	7	Migración, perfiles, persistencia real, vista de entrenamiento, FEN simplificado, jugada duplicada y restricciones de la base.

Resultado: **78 de 78 pruebas pasan.** Las pruebas comunes usan un motor UCI falso, así que no necesitan tener Stockfish instalado; la de Stockfish real se activa con la variable STOCKFISH_TEST_PATH.

4. Qué necesitamos de cada grupo

Grupo	Pedido	Puntos
Diseño Técnico	Sumar al diagrama las clases de servicio y dominio, la firma unificada de getNextMove, el respaldo de la IA y los métodos nuevos de repositorio. Fijar el endpoint de importar.	1, 5, 6, 7, 14
Modelado de Datos	Incorporar al modelo físico status, end_reason, alias, ELO, uci, fen_before_key y la vista v_training_moves (el SQL está en backend/db/migrations/001_init.sql).	2, 3, 4, 8, 9
Arquitectura	Alinear la sección 8 (endpoints y puerto) con Diseño Técnico, el fallback de 6.2 y la prueba de la sección 10.	6, 10, 12
Análisis y Requisitos	Escribir los CU de importar y exportar, y aclarar los alias en el modo local.	3, 7
Simulación	Decidir si usan el adaptador de prueba de Backend (backend/src/engines/StockfishAdapter.js) o el suyo. Armar el servicio Python de la IA con el contrato /predict-move y entrenar desde v_training_moves.	6, 8, 9, 14
Frontend	Llamar a engine-move cuando machineTurn sea true, y manejar PROMOTION_REQUIRED y ENGINE_TIMEOUT.	10
QA	Usar el puerto 3001 y los códigos de error del README para los casos de prueba.	10, 12
GC	Decidir base local o Supabase, y revisar la contraseña publicada en la guía.	13
Coordinación	Llevar al profesor la duda del ELO.	4